

SentinelX — Project Briefing

This document is a full summary of the SentinelX project: what was built, why, what papers informed the decisions, what the technical stack is, and what the real results are. It is intended as a handoff brief for generating a presentation slide.

What the project is

SentinelX is a predictive maintenance and anomaly detection system for industrial environments. It ingests real-time sensor data from machines, predicts failures before they happen, explains each prediction using SHAP, and generates natural-language diagnostic summaries via a local LLM. The full stack includes a trained ML model, a REST API, a database, a dashboard, and a CI/CD pipeline.

The project was built solo, in stages, across January 2026.

The real problem that was solved

Industrial monitoring systems traditionally generate massive numbers of false alerts. Operators develop "alert fatigue" and start ignoring warnings — including real ones. The project started as a failure prediction task, but the core insight (drawn from Paper 3, below) was that **the model accuracy wasn't the actual problem. The trust and reliability of the alert system was.**

The goal shifted: not just to predict failures accurately, but to build a system where operators would actually trust and act on the output.

Papers read and how they influenced the project

Four research papers were read and referenced throughout the project. They are in the [/papers/](#) directory.

Paper 1 — "A Hybrid Machine Learning Approach" (A hybrid machine learning approach.pdf)

- Proposed combining supervised and unsupervised methods for fault detection
- Discussed cross-domain feature fusion (combining mechanical + IT/software sensor signals)
- Influenced: the decision to use both XGBoost (supervised) and an autoencoder (unsupervised) in parallel; the design of cross-domain engineered features like `thermal_efficiency_idx` and `torque_wear_product`

Paper 2 — "Anomaly Detection in Network Traffic Using Advanced ML Techniques" (Anomaly_Detection_in_Network_Traffic_Using_Advanced_Machine_Learning_Techniques.pdf)

- Addressed class imbalance in anomaly detection settings
- Discussed SMOTE and the danger of applying it incorrectly relative to train/validation splits
- Influenced: the decision to apply SMOTE **only inside each CV fold** (not globally before splitting), which is a subtle but important data leakage prevention step; the use of PR-AUC as the primary metric instead of accuracy or standard ROC-AUC

Paper 3 — "EmanPublisher / PrimeAsia paper" (EmanPublisher_26_5955primeasia-6110172.pdf)

- The most directly impactful paper for the project's reframing
- Documented that the **industry average false positive rate is approximately 54%**, and that this rate is what causes alert fatigue and operator distrust
- Provided the benchmark used to compare SentinelX's 1.44% FP rate against
- Influenced: the core thesis of the project (trust > raw accuracy), the decision to implement multiple threshold modes instead of one fixed cutoff, and the inclusion of SHAP explanations as a trust-building mechanism

Paper 4 — "Deep Learning-Based Real-Time Data Quality Assessment" (Deep LearningBased RealTime Data Quality Assessment.pdf)

- Addressed data quality in streaming/real-time contexts
- Influenced: the design of the data ingestion pipeline (`stream_manager.py`) with quality gates that validate incoming sensor data before it reaches the model

What was built, stage by stage

The project was developed in 9 stages, each corresponding to a Jupyter notebook and a set of source files.

Stage 1 — Data ingestion and quality gates Raw sensor data arrives as a stream. A `stream_manager.py` validates incoming readings before they enter the pipeline. Informed by Paper 4.

Stage 2 — Feature engineering 19 raw sensor inputs are transformed into 124 engineered features:

- Rolling statistics (mean, std) over windows of 6, 12, 36 steps
- Lag features (1, 2, 6 steps back) for trend detection
- Cross-domain derived features: `thermal_efficiency_idx`, `torque_wear_product`, `power_anomaly_score`, `error_under_stress` This stage had more impact on model performance than model selection.

Stage 3 — Model training XGBoost classifier trained on the 124 features.

- Class imbalance: ~3% positive (failure) class
- SMOTE applied inside each fold of 5-fold stratified CV only — never on validation data
- Optimized for `aucpr` (PR-AUC), not accuracy
- Early stopping at 50 rounds
- 500 estimators, depth 6, learning rate 0.05
- Final trained on ~411 trees on average across folds

Stage 4 — Anomaly detection baseline A PyTorch autoencoder trained only on healthy machine readings. It learns to reconstruct normal behavior; high reconstruction error signals anomaly. Architecture: 124 input → 16-dimensional latent space. Used in parallel with XGBoost to catch novelty/out-of-distribution events that supervised models might miss.

Stage 5 — Evaluation and threshold calibration Model evaluated at 500k scale. Three operational threshold modes were defined based on precision-recall tradeoffs:

- **ultra_safe**: threshold 0.937, precision 0.90, recall 0.48
- **balanced**: threshold 0.634, precision 0.759, recall 0.827, F1 0.792
- **sensitive**: threshold 0.217, precision 0.614, recall 0.950 Cost-optimal analysis: FP costs \$500, FN costs \$5,000 → projected net savings \$11.6M under cost assumptions.

Stage 6 — SHAP explainability engine SHAP TreeExplainer computes per-prediction feature attributions. Top contributing features across the dataset:

1. disk_io_wait_ms (1.93 mean |SHAP|)
2. error_rate_pct (1.10)
3. error_rate_pct_lag_1 (0.78)
4. fuzzy_pid_output (0.59)
5. packet_loss_pct (0.43)
6. vibration_mm_s (0.34)

Stage 7 — FastAPI backend and PostgreSQL persistence REST API at port 8000. Endpoints: `/predict`, `/stream`, `/health`, `/alerts`, `/fatigue`. PostgreSQL with JSONB columns for sensor payloads and SHAP values — schema is flexible because sensor configurations differ by machine type. SQLAlchemy ORM.

Stage 8 — Streamlit dashboard Frontend showing live predictions, historical alert log, SHAP waterfall charts, and a savings counter comparing SentinelX FP rate to industry baseline.

Stage 9 — Local LLM integration Ollama running Llama 3.2 locally generates natural-language diagnostic summaries from SHAP values. Async aiohttp calls, structured output (JSON + Markdown), chain-of-thought prompting, automatic fallback to rule-based logic if Ollama is unavailable. No cloud API calls — chosen for data privacy, zero latency dependency, zero token cost.

Infrastructure and reliability

- **Docker Compose**: all three services (API, dashboard, PostgreSQL) containerized from the beginning
- **GitHub Actions CI/CD**: lint (Black, Ruff, isort), pytest unit tests, Bandit security scan, Docker build, integration tests with a PostgreSQL service container, model validation, staging deploy on main, production deploy on version tags
- **Playwright E2E tests**: 65 tests covering API endpoints and dashboard UI

CI/CD was implemented from day one, not retrofitted. This was a deliberate engineering decision — reproducible environments and test coverage from the start.

Real metrics (from evaluation_results.json and cv_metrics.json)

Metric	Value
PR-AUC (test set)	0.861
PR-AUC (5-fold CV mean)	0.862
PR-AUC (CV std dev)	±0.010

Metric	Value
F1 (balanced mode)	0.792
Precision (balanced)	0.759
Recall (balanced)	0.827
False Positive Rate	1.44%
Industry FP Rate (Paper 3)	~54%
Projected net savings	\$11.6M (cost model)
Engineered features	124
CV folds	5
Avg trees used	411
Autoencoder latent dim	16
Training samples (healthy, for AE)	37,916

Key engineering decisions and the reasoning behind each

PR-AUC instead of accuracy: The dataset has ~3% positive class. A model that always predicts "no failure" scores 97% accuracy. PR-AUC specifically measures how well the model finds the rare class, so it can't be gamed by predicting the majority.

SMOTE inside CV folds only: A common mistake is to apply oversampling to the entire dataset before splitting into train/validation. This leaks synthetic samples into the validation set and inflates scores. In SentinelX, SMOTE is applied strictly inside each fold's training portion.

Three threshold modes: There is no universally correct threshold. An operator managing safety-critical equipment wants precision (ultra_safe). A system doing pre-screening wants recall (sensitive). Exposing this as a configurable mode instead of a fixed cutoff is more honest about the tradeoff.

Local LLM (Ollama + Llama 3.2): Cloud LLM APIs introduce latency variance, per-token costs, and require sending sensor data to a third party. Running locally with Ollama eliminates all three concerns. The fallback to rule-based logic ensures the system works even when Ollama is down.

PostgreSQL JSONB: Different machine types have different sensor configurations. A rigid relational schema would require migrations every time a new sensor type is added. JSONB stores the raw payload flexibly while still allowing indexed queries.

CI/CD from day one: Not adding tests and Docker "after the project works." Containerized and tested from the beginning means every iteration is reproducible and regressions are caught automatically.

What the project is NOT

- It is not an "AI demo." It has a real backend, real tests, real CI/CD, and real cost/threshold modeling.

- The \$120M savings figure in the README is an extrapolation under broad industry assumptions. The honest number from the actual cost model (FP=\$500, FN=\$5k, evaluated on a 50k sample) is \$11.6M net savings. Do not cite \$120M as a project result.
 - It is not a real-time deployed system. It is a fully functional local stack designed to demonstrate production-grade engineering practices.
-

Context for the audience

This briefing is for generating a technical internship interview presentation slide for a Research Data Engineering team at a biotech/research company. The audience is technical. The goal is to walk through the project in 3–5 minutes while naturally pre-answering likely interview questions (metric choice, leakage, threshold tradeoffs, why local LLM, etc.). The tone should feel like an engineering walkthrough, not a startup pitch.